

# Zuno 企业私有知识 Agent Workspace 目标架构研究报告

## 执行摘要

当前的 Zuno 已经不是“空白仓库”，而是一个有明确边界的本地优先 Agent Workspace：README 明确写出它把 Vue Web、Electron Desktop、FastAPI 后端、当前 `GeneralAgent` 单循环主线、Knowledge/RAG/GraphRAG、工具能力、MCP 语境和本地 Eval 放在同一个 monorepo 中；同时也清楚声明，当前实现虽然已经证明了 query router、五类 memory taxonomy、ToolCard metadata、GraphRAG query contract、trace/evidence/artifact 等基础，但**还不是**生产级 memory retrieval/consolidation、动态 capability orchestration、完整前端 trace panel、成熟的 LLM-first GraphRAG extraction，当前主线仍是 `GeneralAgent` single loop 而不是产品级 LangGraph/多 Agent 运行时。仓库当前前端是 Vue Web + Electron Desktop，后端 runtime 边界在 `src/backend/zuno`。架构 HTML 也明确把 4+1 / View & Beyond 作为“目标架构叙事”，而不是宣称这些都已经实现。<sup>1</sup>

因此，对“Zuno 距离目标架构和八大交付物还有多少差距”的结论可以非常直接地说：**文档治理与交付物治理已经明显成型，运行时与产品能力还处在 foundation / integration 阶段**。换句话说，Zuno 现在已经很像一个“架构与工程规范成熟、运行时能力待产品化”的项目，而不是“GraphRAG、agentic RAG、LangGraph 深度使用、企业级安全与自动化评测都已经系统落地”的项目。README 明确把 `product_mode = normal | enhanced | auto`、`query_method = auto | basic | local | global | drift` 写成“当前基础已证明”；同时又明确写出 production-grade memory、完整 RRF/rerank 治理、产品级动态工具编排、完整前端 trace 面板和多 Agent/coding mode 不能写成 Current。<sup>2</sup>

如果把目标场景收束为“企业内部文档知识库 + 多功能助手”，Zuno 其实有非常强的主叙事优势：本地优先、企业私有文档、GraphRAG 的 local/global/DRIFT、evidence/citation/trace、合同/制度/技术文档/项目知识库这些点天然是一个闭环，而且比“通用聊天 Agent 平台”更聚焦、更适合简历和面试叙事。GraphRAG 官方文档强调，它面向私有非结构化语料，通过知识图谱、社群摘要和 local/global/DRIFT 查询提升对私有知识库的问答与全局综述能力；经典 RAG 论文也把 provenance、可更新显式记忆和知识密集任务作为核心动机。<sup>3</sup>

在后端参考上，`deepsearch-agents` 很适合作为 Zuno 下一阶段的“服务形态参考物”，因为它已经把 `FastAPI + WebSocket + DeepAgents + React`、异步执行、事件推送、文件上传下载和 Orchestrator-Workers 模式放在一个可学习的项目里；它甚至在 README 里直接点名“后续可以继续扩展权限控制、任务队列、事件持久化、评测体系”。这和 Zuno 当前文档里尚未完成的能力几乎一一对位。<sup>4</sup>

基于这些现实边界，本报告给出的目标架构不是“再画一版更复杂的图”，而是把 Zuno 收束成一个可执行的企业私有知识 Agent Workspace 目标：**以 Single Controller Agent 为核心运行时，以 Context/Memory、Tool/Capability、Knowledge/Retrieval、Document Ingestion、Security/Governance、Eval/Observability 六个平面支撑；在模型层采用本地与 API 混合路由；在检索层同时建设 Basic RAG 与 GraphRAG；在工具层区分 capability domain 与 executor/connector；在安全层用审批、授权、脱敏、MCP trust、网络白名单和多层沙箱做硬边界；在评测层用 LangSmith 兼容 trace + RAGAS/DeepEval + CI gate 做持续回归。LangGraph/Deep Agents 提供 durable execution、streaming、human-in-the-loop、subagent、context engineering 等关键基建；MCP 规范为工具发现、schema 与远程授权提供标准接口；LangSmith 则提供 span/run 数据格式、thread/session 归组、离线/在线评测闭环。**<sup>5</sup>

## 当前状态与目标边界

Zuno 自己的文档已经给出了一个非常清晰的“Current/Target/Future”边界。README 把项目一句话目标写成“Local-first Agent Workspace = Single Controller Agent + Context/Memory Engine + Capability/Tool Retrieval + Knowledge/GraphRAG Retrieval + Evidence/Citation/Trace/Eval + Typed API + Web/Desktop”，但紧接着又强调“这里的 Single Controller Agent 是目标架构角色；当前实现主线是 GeneralAgent single loop”。这意味着，Zuno 现在最成熟的并不是“运行时能力本身”，而是“架构边界意识”——它知道什么已经实现，什么只是目标。 <sup>2</sup>

README 还进一步明确：当前 GraphRAG/Knowledge 主线已经有 KnowledgeQueryService、GraphRAGQueryService、GraphRAGProjectSnapshot 和 KnowledgeQueryResult；当前 Query Router foundation 已贯通 product\_mode 与 query\_method；当前 Context/Memory foundation 已证明 Context Pack policy、source id coverage、五类记忆 taxonomy、review contract 和轻量 readback；当前 Capability foundation 已证明 ToolCard compact metadata、Native BM25 ToolCard retrieval、MCP/local tool policy trace 和 capability selection trace bridge；但这些都还不是 production-grade memory retrieval/consolidation、完整动态工具编排或完整 GraphRAG extraction/rerank 治理。也就是说，Zuno 现在已经有“骨架”和“契约”，但还缺“肌肉”和“器官级联动”。 <sup>2</sup>

与此相对，deepsearch-agents 展示了另一种更“产品化”的形态：它直接把 FastAPI、WebSocket、DeepAgents、React、异步执行和事件推送组合成一个前后端实时闭环，并用 Orchestrator-Workers 模式做多专家协作。这个项目并不意味着 Zuno 一定要抄成多 Agent；它真正有价值的地方在于：它告诉我们，一个成熟 Agent 后端通常至少要具备持久 runtime、实时事件流、异步后台任务、文件产物、可中断会话、前端联动这些基础能力。 <sup>4</sup>

从场景上看，Zuno 最应该收束的不是“万能 Agent 平台”，而是“企业内部文档知识库 + 多功能助手”。这里的“多功能”不是放弃主叙事，而是围绕企业知识做扩展动作：问答、制度解释、文档比较、摘要报告、知识发现、邮件草拟与发送、表单填充、知识库检索、项目文档查询、合同条款对照、简历与人才档案检索。这一叙事同时兼容“企业知识库”“个人知识/简历资料”“部门项目资料”三类资产，而且最适合本地优先与私有部署。GraphRAG 文档强调其针对私有文本语料的 structured, hierarchical retrieval；MCP 把数据库、API、文件系统等能力统一成标准工具接口；Deep Agents 把 planning、subagent、filesystem context 和 long-running tasks 组合成企业任务型 Agent 的标准骨架。 <sup>6</sup>

基于这些边界，本报告采用以下假设：基础设施供应商无特定约束；允许本地模型与 API 模型混合；目标前端按你的要求以 React + Vite 为 Target 讨论，但会明确指出这与当前 Vue/Electron Current 不同；目标后端以 FastAPI + LangGraph/Deep Agents runtime + task queue + sandbox cluster 为核心；知识平面同时保留 Basic RAG 与 GraphRAG，两者由 router 决策，不把 GraphRAG 当成所有问题的默认答案。这个目标并不与当前仓库冲突，因为 Zuno 的 README 已经明确允许 Target 继续扩展，而不把这些目标伪装成 Current。 <sup>7</sup>

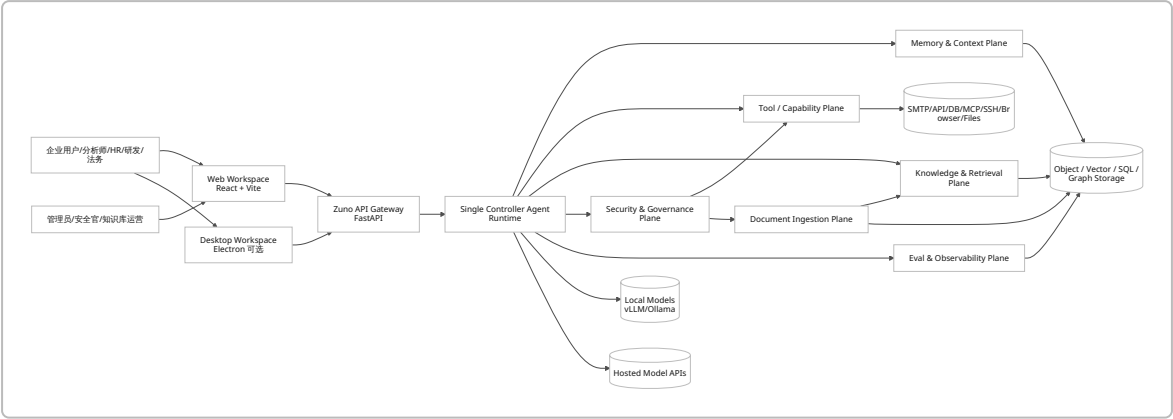
## 目标架构总览

目标架构建议采用“单控制器运行时 + 多平面支撑”的方式，而不是先升级成默认多 Agent。这个选择与 Zuno 当前架构文档是一致的：现有 architecture.html 的逻辑视图已经把系统画成 Frontend -> Backend API -> Single Controller Agent -> (Context and Memory / Capability and Tool / Knowledge and Retrieval) -> Evidence/Citation/Trace/Eval -> Platform Infrastructure，并明确写着“默认主线是 Single Controller Agent；Memory、Tool、Knowledge 是能力层，不是默认多 Agent runtime”。换言之，规划模块不应该被画成独立于 Agent 核心之外的“第五脑袋”，而应该被嵌入 Single Controller Agent runtime 内部，作为其 plan/react/reflect/replan/reflexion 的状态机能力。 <sup>8</sup>

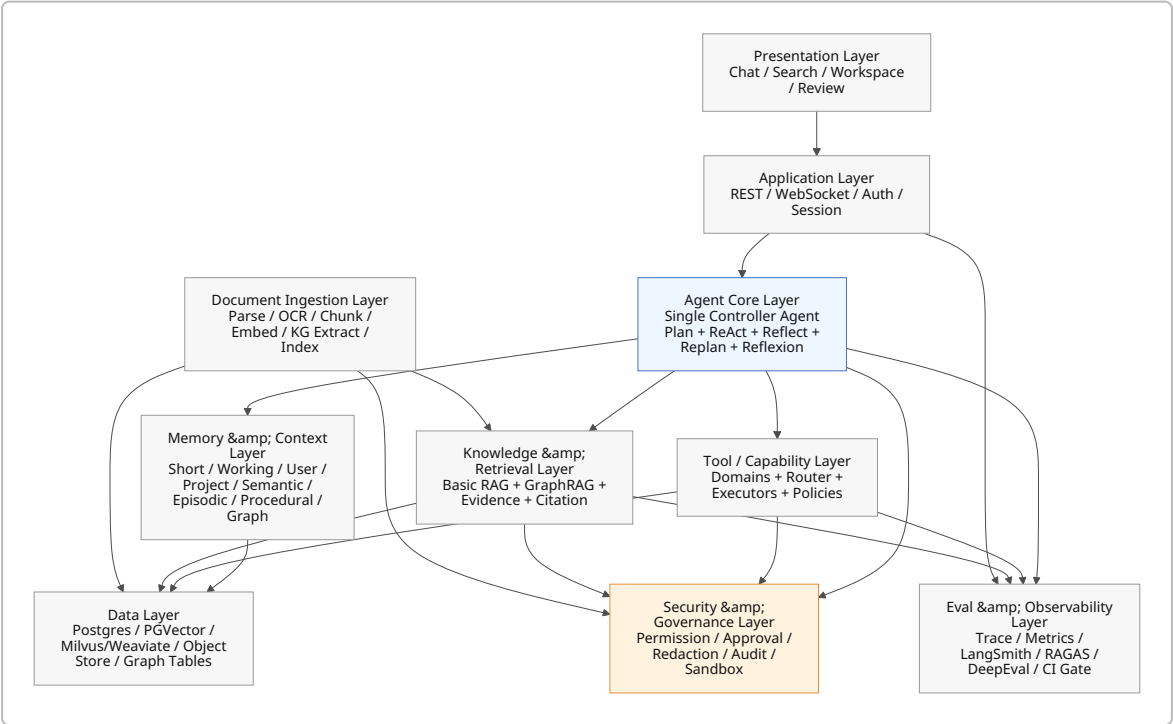
这套目标架构同时综合了 LangGraph 的 durable execution / persistence / HITL、Deep Agents 的 planning / filesystem context / subagents / long-term memory、GraphRAG 的 structured hierarchical retrieval、MCP

的标准工具 schema 与远程授权、LangSmith 的 run/span/thread/eval 体系。下面十张图不是“当前已经实现”，而是本报告建议 Zuno 收口后的 **Target Architecture 视图集**。这些图的复杂度会显著高于当前 architecture.html，但仍然保持“可落地”的层次。 9

图一：系统上下文

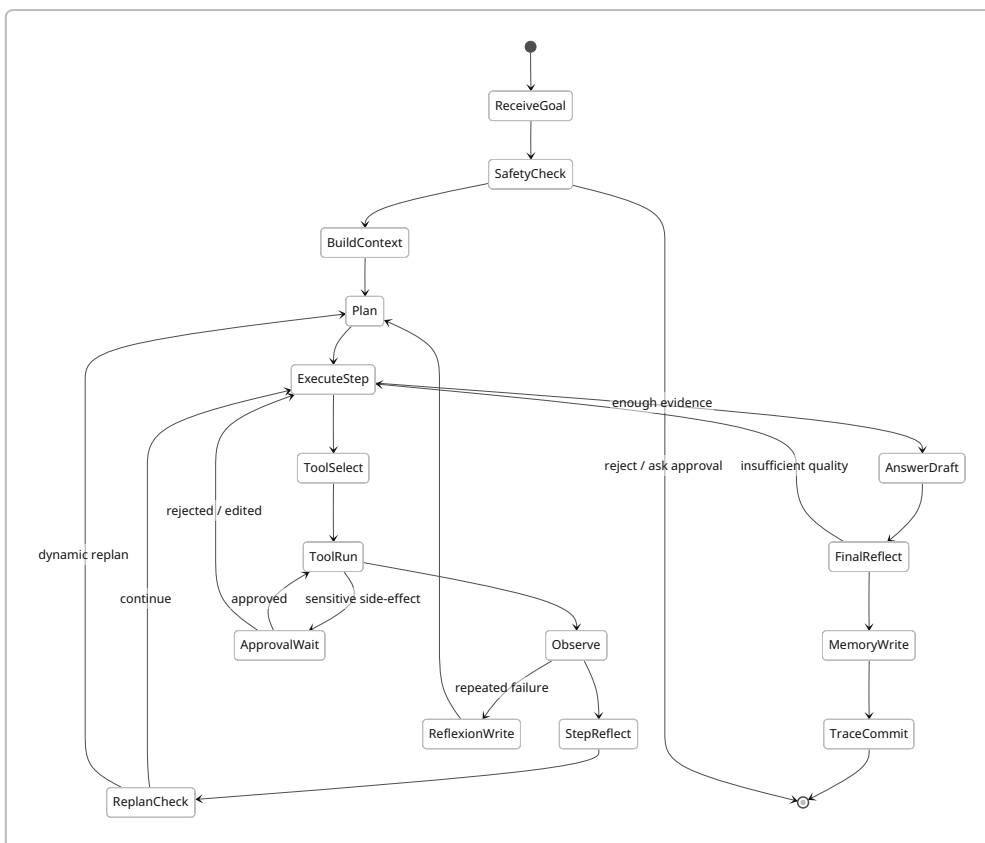


图二：逻辑分层架构



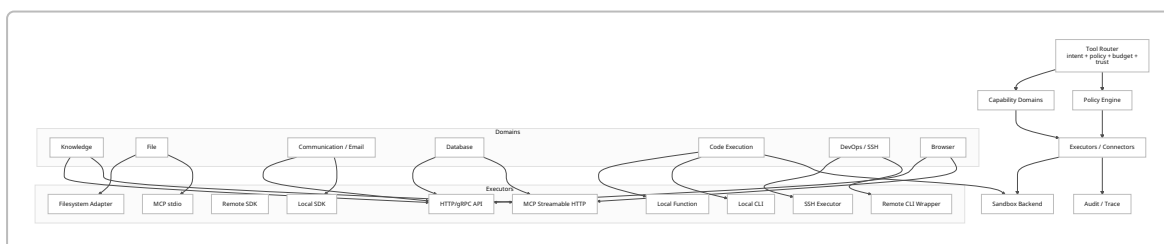
图三：Single Controller Agent 运行时状态机

这张图把“规划模块是不是就是 react+plan+reflection”的问题落成状态机答案：是，但不应该拆成孤立模块，而应该作为控制器运行时内部的多个状态与策略。ReAct 负责单步推理-行动循环，Plan-and-Execute 负责全局任务分解，Reflection 负责当前轮或终局质检，Dynamic Replan 负责中途修正计划，Reflexion 负责把失败经验沉淀进可复用记忆。其理论来源分别对应 ReAct、Plan-and-Solve、Reflection 等论文；运行时承载则建议交给 LangGraph/Deep Agents。 10



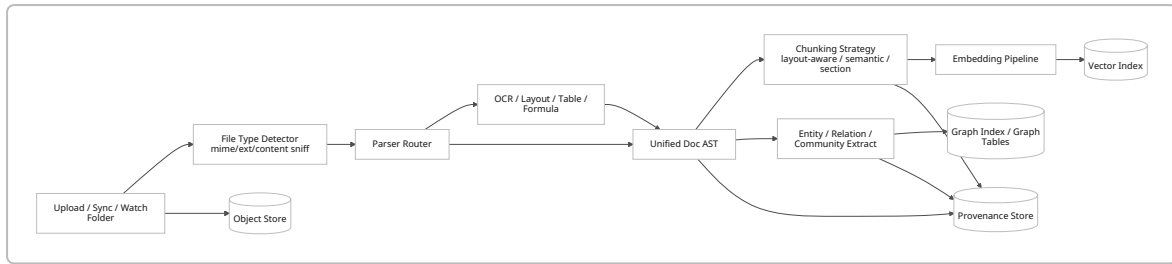
图四：工具能力平面与执行器

MCP 官方规范把工具定义为带名称、元数据和输入 schema 的可调用能力，当前标准传输是 `stdio` 与 `Streamable HTTP`；Deep Agents 也明确支持 custom functions、LangChain tools 和 MCP server tools。基于这个现实，工具层最适合分成两层：上层是 **capability domains**（知识、文件、邮件、数据库、代码执行、DevOps/SSH、浏览器等），下层是 **executors/connectors**（local function、SDK、API、CLI、SSH、MCP、sandbox filesystem）。这样“邮件”“文件”“代码执行”是能力域，不与 API/SDK/CLI/MCP 混为一类。 11



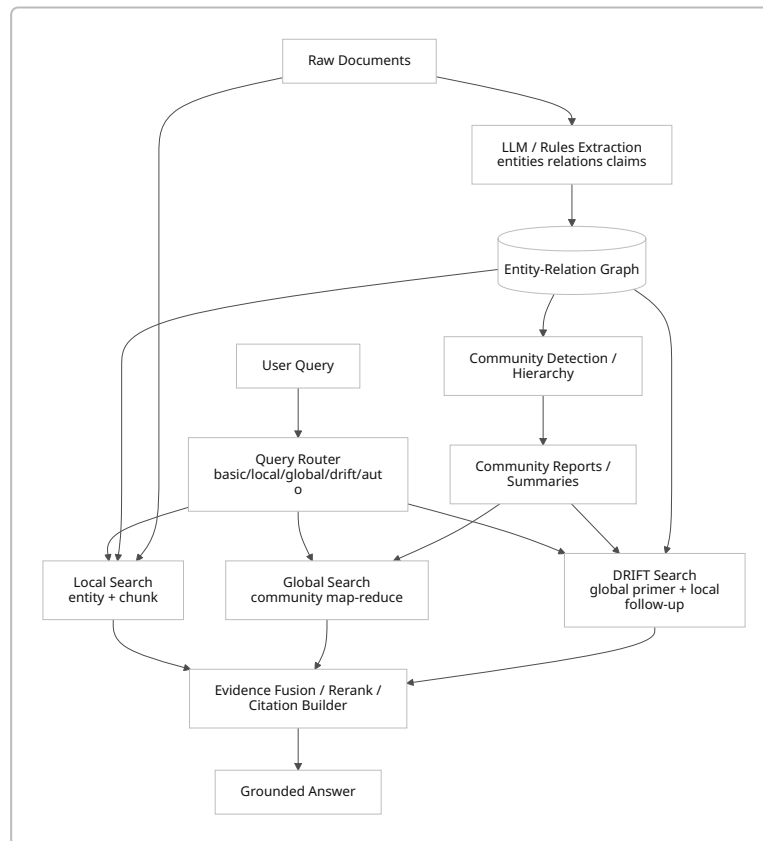
图五：文档摄取流水线

Docling 官方与 LangChain 集成都强调其对 PDF/DOCX/PPTX/HTML 等格式的统一、结构化解析；Unstructured 官方列出大量支持格式，包括 pdf、docx、pptx、txt、md、html、eml、image、xlsx 等；PaddleOCR 与其 layout parsing 文档则提供图片/PDF OCR、版面区域分析、表格/公式/阅读顺序识别。基于这些能力，Zuno 的文档摄取层应当是“parser router + layout/OCR + chunk + embed + entity/relation extraction + dual index + provenance store”的流水线，而不是“单一 PDF->文本”转换器。 12



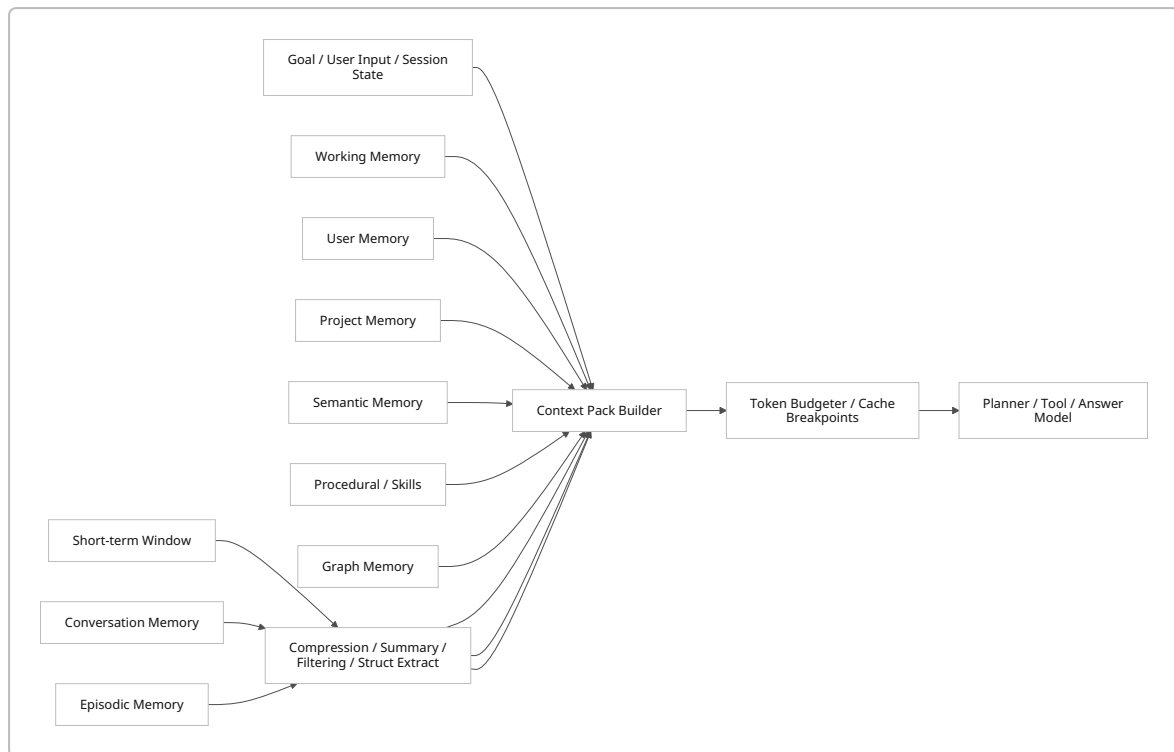
图六：GraphRAG 架构

微软 GraphRAG 明确把流程定义为：从原始文本抽取知识图谱、建立社群层级、生成 community summaries，然后在查询时按问题类型走 local、global、DRIFT 等不同路径。其中 global search 使用 community reports map-reduce；local search 把图谱结构化数据与原文 chunk 结合；DRIFT 在 global 与 local 之间动态穿梭。这个形态很适合企业内部文档，因为“全局主题 / 部门共识 / 实体关联 / 细节证据”天然同时存在。<sup>13</sup>



图七：记忆与 Context Builder

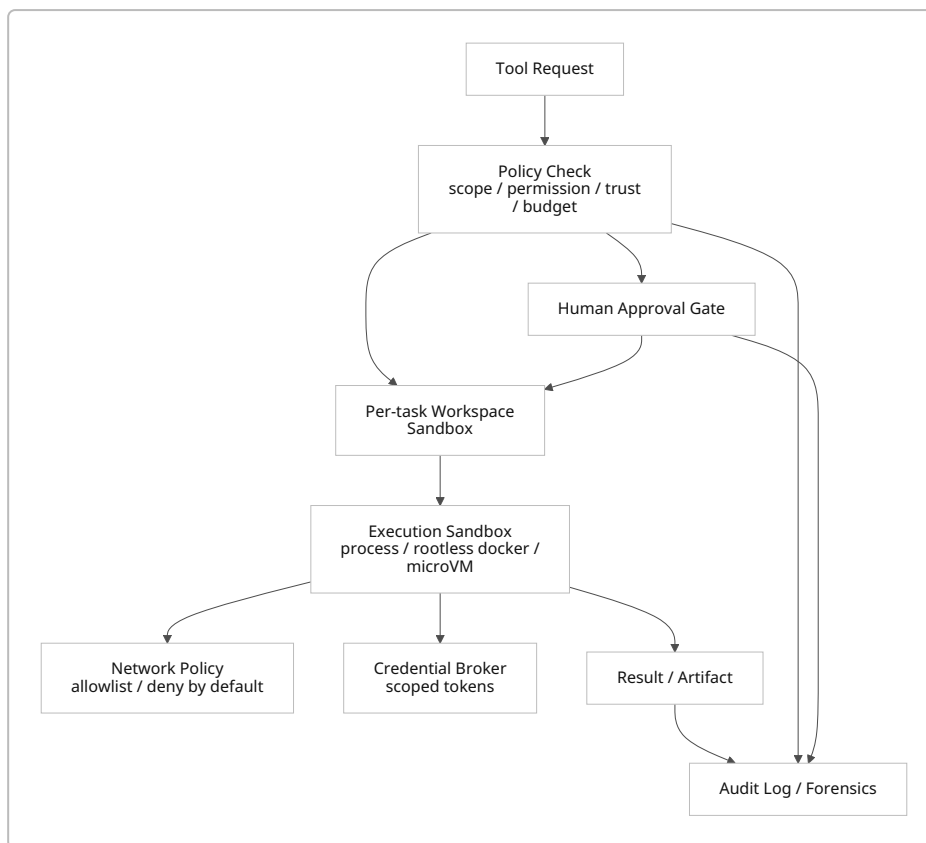
CoALA 提出 language agent 可借鉴 modular memory 结构；Generative Agents 用 relevance、recency、importance 做记忆检索并通过 reflection 形成高层总结；Deep Agents 和 LangChain 则把 context engineering 直接定义为“把正确的信息和工具以正确格式放进上下文”的核心工作。对 Zuno 来说，Context Builder 不只是“拼 prompt”，而是一个独立的装配器：输入会话、用户、项目、任务、工具、知识与压缩记忆，输出一个有 token 预算、来源标注、风险标签和缓存边界的 model-visible context pack。<sup>14</sup>



图八：沙箱与策略执行

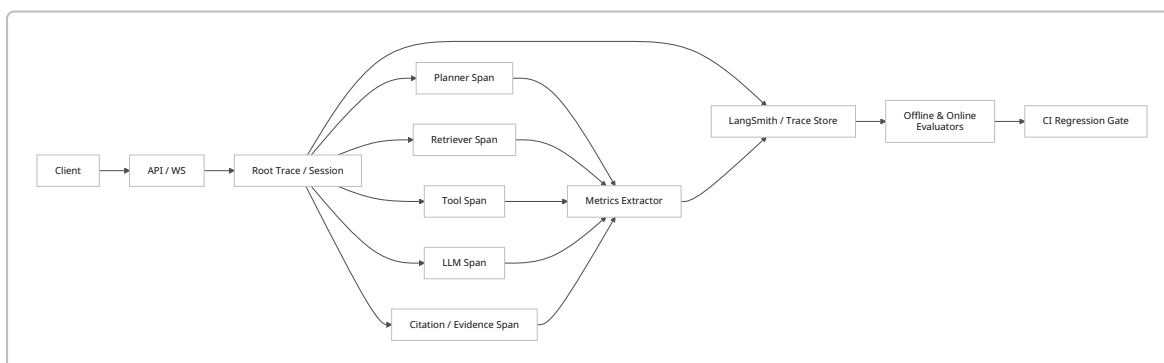
Deep Agents 文档清楚写明：当代理要写文件、安装依赖、执行 shell 命令时，应把这些能力放进 isolated sandbox；Docker rootless 可以降低 daemon 与 runtime 的 root 风险；gVisor 提供更强的 userspace kernel 隔离；Firecracker 和 Kata Containers 则提供基于硬件虚拟化的更强隔离边界。对企业知识助手而言，最成熟的做法不是“所有东西都在沙箱里”，而是三层：**policy sandbox（先决策）**、**workspace sandbox（工作区隔离）**、**execution sandbox（代码/命令执行隔离）**。

15



图九：评测与可观测性链路

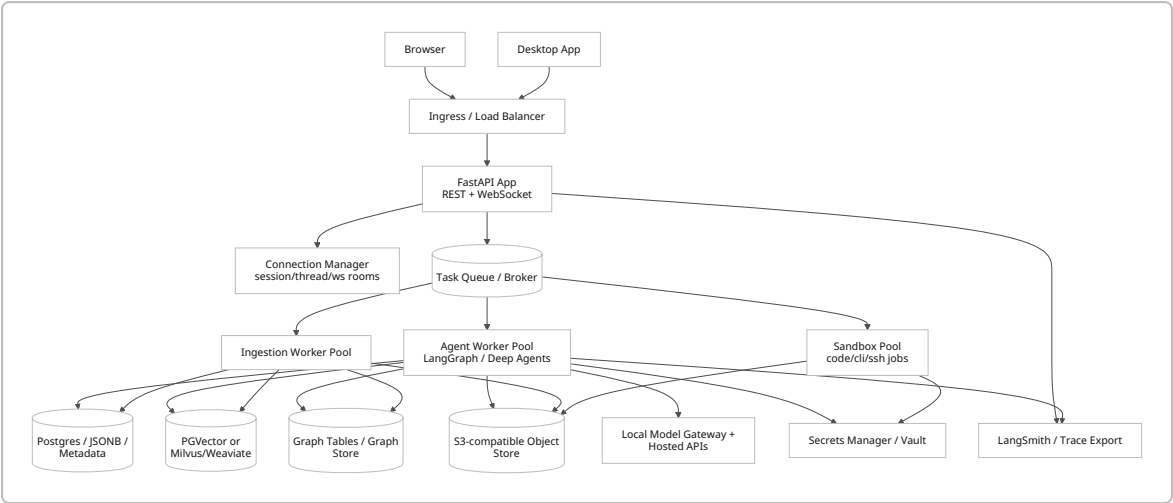
LangSmith 的 run/span 数据格式明确包含 `trace_id`、`run_type`、inputs/outputs、events、tags、token/cost、session/thread 等字段，并支持 offline/online evaluation、dataset、experiments、filters、sampling 与 production trace feedback loop。RAGAS 提供 faithfulness、context recall、context precision 等 RAG 指标；DeepEval 提供 answer relevancy、faithfulness、contextual precision/relevancy、JSON correctness 和 G-Eval。对 Zuno 来说，最关键的是把这些东西统一成一条 trace 主线，而不是把“自动评测”另做成一个孤岛。<sup>16</sup>



图十：部署拓扑

FastAPI 官方文档适合承载 typed API 与 WebSocket；Celery 官方文档强调其多 worker、broker、横向扩展能力；Deep Agents/LangGraph 提供长任务编排、subagent 与 persistence；LangSmith 提供云、混合或 self-hosted 的观测/评测路径。综合这些现实，一个成熟的 Zuno 拓扑应当是：API 网关 + 会话/连接管理 + runtime

workers + ingest workers + sandbox workers + object/vector/sql stores + optional LangSmith/export path。 17



核心运行时、记忆与工具

模型层建议采用**混合路由而非单模型神教**。Deep Agents 文档说明其适配任何支持 tool calling 的 LangChain chat model，并且可以在运行时选择模型；vLLM 官方提供 OpenAI-compatible server，适合在内网用统一接口服务本地模型；Ollama 官方则默认把模型 API 暴露在本机 localhost 上，适合开发机或轻量部署；LiteLLM 提供路由、重试、fallback、预算与 observability callback，适合作为统一模型网关。对于企业私有知识助手，这意味着最稳妥的策略不是“全本地”或“全 API”，而是按任务分层：planner/judge/reflection 可走更强模型，retrieval query rewrite/tool selection/main answer 根据预算与隐私等级在本地与 API 间切换，embedding/reranker/OCR-VLM 则各用专门模型。 18

下面这张表是建议的模型分工，不是当前实现现状。

| 模型角色                | 主要职责             | 推荐部署                   | 路由策略        | 关键约束            |
|---------------------|------------------|------------------------|-------------|-----------------|
| Planner / Replanner | 任务分解、动态重规划       | 强 API 模型或高配本地模型        | 仅在复杂任务触发    | 允许较高延迟，需强工具调用质量 |
| Tool Selector       | ToolCard 检索、参数草拟 | 本地中型模型优先               | 高并发、低成本     | 需结构化输出稳定        |
| Answer Model        | 企业文档问答、报告生成      | 本地优先，超长复杂问题可走 API      | 依据保密级别与预算切换 | 必须引用证据          |
| Judge / Reflection  | 事实性、格式、引用覆盖检验    | 强模型或专用 judge 模型        | 只在关键节点启用    | 成本可控、必须可追踪      |
| Embedding           | 文本/段落向量化         | 本地服务优先                 | 批处理         | 模型与索引兼容         |
| Reranker            | 二阶段精排            | 本地 cross-encoder 或 API | top-N 后使用   | 对召回和延迟折中        |
| OCR/VLM             | 图像/扫描件/表格理解      | 本地文档 VLM 优先            | 仅对复杂文件触发    | 成本高，应按需启用       |

模型路由建议的数据契约如下，这里是本报告的建议 schema：



```
{
  "route_class": "planner|tool_selector|answer|judge|embedding|reranker|ocr_vlm",
  "privacy_tier": "local_only|hybrid_ok|external_ok",
  "latency_budget_ms": 6000,
  "cost_budget_usd": 0.02,
  "preferred_provider": "local|openai|anthropic|google|custom",
  "fallback_chain": ["local:qwen", "api:gpt", "api:claude"],
  "requires_json_schema": true,
  "prompt_cache_scope": "system|tools|history_prefix"
}
```

在记忆层，建议把 Zuno 的当前“五类记忆 taxonomy”扩展成一个更完整的企业工作流记忆模型：`short/working/conversation/project/user/semantic/episodic/procedural/graph`。CoALA 给出了模块化 memory 的理论基础；Generative Agents 证明 relevance、recency、importance 三因素与 reflection summary 的有效性；Deep Agents 的 context engineering 与 Code memory/skills 则证明 thread-scoped memory、cross-session memory、AGENTS.md/skills 这类结构化上下文在工程上可行。对企业知识助手而言，最重要的不是“记更多”，而是“按作用域和职责记对”。<sup>19</sup>

建议的记忆分层职责如下：

| 记忆层          | 责任                | 存储                             | 读取时机                            | 写入策略                  |
|--------------|-------------------|--------------------------------|---------------------------------|-----------------------|
| Short-term   | 最近消息窗口、工具回显       | session state / checkpoint     | 每轮必读                            | 自动                    |
| Working      | 当前任务 TODO、临时推理中间态 | checkpoint / thread scratch    | 每步必读                            | 自动、可清理                |
| Conversation | 会话摘要、关键决策         | SQL + summary blob             | 多轮对话续写                          | 滑窗阈值触发摘要              |
| Project      | 项目级约束、术语、资料入口     | SQL/JSONB + files              | 项目作用域任务                         | 明确确认后写入               |
| User         | 偏好、角色、审批习惯        | SQL/JSONB                      | 用户复用场景                          | 审批确认后写入               |
| Semantic     | 事实知识、术语定义         | vector + metadata              | 事实问答/召回                         | 结构化抽取后写入              |
| Episodic     | 某次任务经历与结果         | SQL/event log + vector         | 类似任务重现                          | 以事件和总结双写              |
| Procedural   | workflow、SOP、技能   | skill files / markdown / graph | 需要执行步骤时                         | 人工审核优先                |
| Graph        | 实体关系、社区摘要         | graph tables/store             | GraphRAG / dependency reasoning | ingestion pipeline 生成 |

记忆压缩不应只做“滑动窗口”。OpenAI 和 Anthropic 的 prompt caching 文档都说明，重复前缀可以显著降低输入成本和延迟；RAGAS/DeepEval 生态也提醒我们，长上下文会直接影响评测与质量成本。因此建议 Zuno 同时采用七种手段：**滑动窗口、摘要压缩、层级摘要、重要性过滤、结构化抽取、主动压缩、prompt**

**caching**。其中主动压缩应优先用于冗长工具输出，prompt caching 则放在 system prompt、ToolCard schema、稳定记忆前缀和最近历史边界上。 20

建议的 Context Builder 接口如下：

```
{
  "thread_id": "uuid-v7",
  "session_id": "uuid-v7",
  "goal": "帮我比较员工手册和新合同中的保密条款差异",
  "scope": {
    "user_id": "u_123",
    "project_id": "proj_hr_handbook",
    "workspace_id": "ws_hr"
  },
  "requested_tools": ["knowledge.search", "file.read", "email.send_draft"],
  "privacy_tier": "local_only",
  "token_budget": {
    "max_input_tokens": 24000,
    "reserved_for_output": 4000
  },
  "pack_sections": [
    "system_policy",
    "task_plan",
    "working_memory",
    "project_memory",
    "retrieved_evidence",
    "tool_schemas",
    "user_constraints"
  ]
}
```

规划/运行时建议采用**单控制器 + 五机制叠加**，但把它们实现成一个统一状态机，而不是五个互相抢权的 agent loop。ReAct 负责每一步 reason-act-observe；Plan-and-Execute 负责前置 plan；Dynamic Replan 在“证据不足/工具失败/任务转向”时回到 plan；Reflection 在 step 级和 final 级分别质检；Reflexion 则把错误经验进入 episodic/procedural buffer，供下次决策参考。LangGraph 的 shared state、discrete nodes、durable execution、interrupt/HITL 能很好承载这一设计。 21

下面是建议的运行时组件契约矩阵，属于本报告的目标设计。

| 组件              | 责任                       | 接口                                     | 数据契约                           | 存储       | 安全控制                 | 测试 / 评测   |
|-----------------|--------------------------|----------------------------------------|--------------------------------|----------|----------------------|-----------|
| Session Manager | 会话生命周期、thread/session 归组 | POST /v1/sessions、WS /v1/sessions/{id} | session_id, thread_id, user_id | Postgres | auth、workspace scope | 会话恢复、断线重连 |

| 组件               | 责任                       | 接口                                       | 数据契约                           | 存储                      | 安全控制                                | 测试 / 评测                          |
|------------------|--------------------------|------------------------------------------|--------------------------------|-------------------------|-------------------------------------|----------------------------------|
| Context Builder  | 组装 model-visible context | runtime event <code>context.build</code> | <code>ContextPack</code>       | Postgres + cache        | PII redaction、budget guard          | token budget、source coverage     |
| Planner          | 任务分解与重规划                 | runtime event <code>plan.created</code>  | <code>TaskPlan/TaskStep</code> | checkpoint              | 高风险任务需 approval flag                | step coverage、plan stability     |
| Executor Loop    | ReAct 推理与工具驱动            | runtime event <code>step.*</code>        | <code>RuntimeState</code>      | checkpoint              | tool policy gate                    | deterministic loop tests         |
| Reflection Judge | step/ final 自检           | <code>eval.reflect</code>                | <code>ReflectionVerdict</code> | trace store             | no side-effect                      | faithfulness / citation coverage |
| Memory Writer    | 记忆筛选与持久化                 | <code>memory.commit</code>               | <code>MemoryRecord</code>      | Postgres/ vector/ graph | write approval for durable memories | leakage / duplication tests      |
| Trace Bridge     | 统一 trace/ span/ events   | <code>trace.emit</code>                  | <code>TraceEvent</code>        | LangSmith / SQL         | immutable audit                     | trace completeness               |

工具层建议采用 **ToolCard Registry + Tool Router + Executor Adapters**。ToolCard 不等于运行时代码；它是 Agent 可见的工具目录项，用于 discovery、policy、budget、approval 与 explainability。它既可以映射本地函数，也可以映射 SDK/API/CLI/MCP/SSH。这个形态与 MCP 的“工具有名称、描述、schema”，以及 Deep Agents 的 tool lists / MCP integration 很一致。 <sup>22</sup>

推荐的 ToolCard 字段如下：

```
{
  "tool_id": "email.send_draft",
  "name": "send_email_draft",
  "display_name": "发送邮件草稿",
  "domain": "communication",
  "description": "向企业邮件系统提交邮件草稿，支持先创建草稿再发送",
  "connector_type": "api",
  "executor_type": "http_api",
  "transport": "https",
  "input_schema": {},
  "output_schema": {},
  "tags": ["email", "side_effect", "approval_required"],
  "read_only": false,
  "side_effect_level": "external_write",
  "approval_mode": "required",
}
```

```

"trust_tier": "tier1_internal",
"auth_scope": ["mail.send", "mail.draft"],
"network_policy": "allow_internal_mail_only",
"budget_policy": {
  "timeout_sec": 20,
  "max_retries": 1,
  "max_calls_per_turn": 1
},
"observability": {
  "log_inputs": "redacted",
  "log_outputs": "metadata_only",
  "emit_span": true
},
"healthcheck": {
  "enabled": true,
  "path": "/health"
},
"examples": [
  {
    "input": {"to": ["hr@example.com"], "subject": "面试安排", "body_markdown": "..."}
  }
]
}

```

一个更完整的 ToolCard JSON 示例如下：

```

{
  "tool_id": "file.search_workspace",
  "name": "search_workspace_files",
  "display_name": "搜索工作区文件",
  "version": "1.0.0",
  "domain": "file",
  "description": "在当前 workspace 范围内搜索文档、代码和 markdown 文件，返回文件元数据与片段。",
  "connector_type": "filesystem",
  "executor_type": "local_function",
  "transport": "inproc",
  "workspace_scoped": true,
  "input_schema": {
    "type": "object",
    "properties": {
      "query": {"type": "string"},
      "paths": {"type": "array", "items": {"type": "string"}},
      "top_k": {"type": "integer", "minimum": 1, "maximum": 50}
    },
    "required": ["query"]
  },
  "output_schema": {
    "type": "object",
    "properties": {

```

```

"results": {
  "type": "array",
  "items": {
    "type": "object",
    "properties": {
      "path": {"type": "string"},
      "snippet": {"type": "string"},
      "score": {"type": "number"},
      "mime": {"type": "string"}
    },
    "required": ["path", "snippet", "score"]
  }
},
"required": ["results"]
},
"tags": ["retrieval", "workspace", "read_only"],
"aliases": ["search files", "find docs"],
"read_only": true,
"side_effect_level": "none",
"approval_mode": "none",
"trust_tier": "tier0_local",
"auth_scope": ["workspace.read"],
"network_policy": "deny_all",
"budget_policy": {
  "timeout_sec": 5,
  "max_retries": 1,
  "max_calls_per_turn": 3
},
"observability": {
  "emit_span": true,
  "capture_args": "safe",
  "capture_result": "metadata_plus_preview"
}
}

```

Tool Router 的选择逻辑建议按以下顺序执行：**作用域过滤** → **权限过滤** → **trust tier 过滤** → **side-effect/approval 过滤** → **cost/latency budget 过滤** → **domain relevance ranking** → **schema fit check** → **health status** → **final pick**。其中“API/SDK/CLI/MCP/SSH”是执行器，“文件/邮件/数据库/代码执行”是能力域；这能避免你之前提到的分类混乱。比如“邮件发送”不是第五种执行器，而是 `communication` 域上的一个 capability，底层可以走 API、SDK、MCP 或 CLI；“文件读写”是 `file` 域上的 capability，底层可以走本地函数、filesystem adapter 或 MCP server。这个分法更适合 Agent 运行时做 explainability 与 policy。其安全边界也与 Deep Agents 的“trust the LLM only at tool/sandbox boundary, not by self-policing”原则一致。

23

## 知识、解析、安全与评测

知识平面应当分成两条主线：**Basic RAG** 和 **GraphRAG**。Basic RAG 解决精确片段问答、法规条款对照、合同/手册/项目文档的局部问题；GraphRAG 解决跨文档主题、实体关系、组织性的全局问题。Lewis 等人的原始 RAG 论文把显式非参数记忆、事实可追溯和知识更新作为核心价值；微软 GraphRAG 则进一步把“从局部到全

局”的私有知识问答补齐。对于企业知识库，最合理的 router 不是默认走 GraphRAG，而是：`basic` 处理局部检索，`local` 处理实体问题，`global` 处理全局概览，`drift` 处理需要从全局线索逐步下钻的复杂问题，`auto` 只是 router 决策，不是第五类执行模式——这一点也与 Zuno 当前 README 完全一致。 <sup>24</sup>

检索建议采用“粗召回 + 精排 + 证据裁决”三阶段。Weaviate 官方把 hybrid search 定义为同时运行向量搜索与 BM25 并融合结果；Milvus 官方支持 dense/sparse/multi-vector hybrid、metadata filtering 与 hybrid reranking；Cohere 等 rerank 产品文档则强调 rerank 通过把最相关文档前置来减少噪声与 token 消耗。对于 Zuno，这意味着企业知识检索不应只依赖 embedding，也不应只靠 GraphRAG。推荐基线是：BM25 + dense vector hybrid 召回，必要时加入 multi-query / query rewrite，再对 top-N 做 cross-encoder rerank，最后做 evidence dedupe 和 citation builder。 <sup>25</sup>

证据与引用模型建议明确到数据契约层。建议核心 DTO 如下：

```
{
  "evidence_id": "ev_01",
  "source_type": "doc_chunk|graph_entity|graph_community|tool_output",
  "source_id": "doc:handbook#p12:c4",
  "document_id": "handbook_v3",
  "locator": {
    "page": 12,
    "section": "保密义务",
    "char_range": [120, 280]
  },
  "content_preview": "员工不得向外部披露公司商业秘密……",
  "retrieval_score": 0.87,
  "rerank_score": 0.93,
  "used_by_answer": true,
  "claim_ids": ["clm_2", "clm_5"],
  "provenance": {
    "ingest_run_id": "ing_123",
    "parser": "docling",
    "chunk_strategy": "layout_semantic_v2"
  }
}
```

文档解析平面应当尽量覆盖企业常见格式：`pdf/docx/pptx/md/txt/html/code/images` 是最低优先级集合，第二梯队可加 `xlsx/csv/eml/msg/epub/rtf`。Docling 适合做高质量 PDF/Office/HTML 统一 AST；Unstructured 适合做广格式兜底；PaddleOCR/PaddleX 适合扫描件、表格、公式、版面、阅读顺序和多语言 OCR。因此，最稳妥的 parser router 不是“只选一个库”，而是：Docling 作为主解析器，失败或不支持时回落 Unstructured，对图片与扫描件触发 PaddleOCR layout 路线。 <sup>26</sup>

建议的 Document Ingestion Plane 组件矩阵如下：

| 组件               | 责任                            | 接口                                  | 数据契约                          | 存储                | 安全控制                           | 测试 / 评测                  |
|------------------|-------------------------------|-------------------------------------|-------------------------------|-------------------|--------------------------------|--------------------------|
| File Intake      | 上传、同步、目录监听                    | <code>POST /v1/ingest/files</code>  | <code>IngestJob</code>        | Object Store      | MIME allowlist、virus scan hook | 大文件、重复上传、断点续传            |
| Parser Router    | 选择解析器与 OCR 路线                 | job event <code>parser.route</code> | <code>ParseRequest</code>     | Postgres metadata | 文件类型白名单                        | 格式覆盖、失败回退                |
| Doc Parser       | 结构化解析                         | <code>parse(file_ref)</code>        | <code>DocumentAST</code>      | Object + AST blob | path sandbox                   | AST golden tests         |
| OCR/ Layout      | 图像/扫描/表格                      | <code>ocr(file_ref, pages)</code>   | <code>LayoutAST</code>        | Object + cache    | VLM/network isolation          | OCR CER/ WER、表格抽取率       |
| Chunker          | layout-aware / semantic chunk | <code>chunk(ast)</code>             | <code>ChunkRecord[]</code>    | SQL/ Vector       | PII tagging                    | chunk overlap/ coverage  |
| Embedding        | 批量向量化                         | <code>embed(chunks)</code>          | <code>EmbeddingBatch</code>   | Vector DB         | local-only if sensitive        | latency、recall@k         |
| KG Extractor     | entity/ relation/ community   | <code>extract_kg(ast)</code>        | <code>GraphFact[]</code>      | Graph tables      | claim confidence threshold     | entity precision/ recall |
| Provenance Store | ingest lineage                | auto                                | <code>ProvenanceRecord</code> | Postgres/ JSONB   | immutable append log           | source roundtrip         |

向量层选型没有唯一答案，但官方文档已经把几个主流选项的边界说得很清楚：FAISS 是高效相似搜索库，适合单机/嵌入式索引；Weaviate 有成熟 hybrid search、BM25、filter 与 rerank 接口；Milvus 支持丰富 ANN 索引、dense+sparse/multi-vector hybrid、filter 和 reranking；pgvector 则允许在 PostgreSQL 内直接管理向量，HNSW 与 IVFFlat 都有官方支持。对 Zuno 来说，如果你要本地优先、部署简单、同时保存业务 metadata，**首选 Postgres + JSONB + pgvector**；如果你预计企业知识库规模、并发和 hybrid 检索会继续扩大，再升级到 Milvus 或 Weaviate 更合适。 <sup>27</sup>

| 方案       | 优点                             | 代价                       | 适合 Zuno 的阶段 |
|----------|--------------------------------|--------------------------|-------------|
| FAISS    | 极简、快、单机好用                      | 不是完整数据库，metadata/服务化要自己补 | 原型、本地 demo  |
| pgvector | 与业务 SQL/JSONB 同库，事务和权限天然统一     | 大规模 ANN 与混合检索能力不如专门向量库灵活 | 当前到中期最合适    |
| Weaviate | Hybrid/BM25/filter/rerank 体验成熟 | 额外基础设施与运维                | 中后期企业检索平台   |

| 方案     | 优点                               | 代价        | 适合 Zuno 的阶段 |
|--------|----------------------------------|-----------|-------------|
| Milvus | 大规模 ANN、hybrid、多向量、reranking 能力强 | 组件更多，运维更重 | 大规模知识库与高并发  |

安全平面必须被当作一等公民，而不是最后贴几条 guardrail。OWASP 把 Prompt Injection 列为 LLM/GenAI 应用的首要风险，Cheat Sheet 明确建议做输入验证、输出校验、远程内容隔离和持续安全测试；NIST 的 Generative AI Profile 也把 prompt injection、数据隐私、系统滥用等列入 AI 风险管理范畴。MCP 官方又强调了 OAuth 2.1 授权、token audience 校验与安全最佳实践。这些都说明：**企业 Agent 的安全边界应该放在输入、工具、网络、凭据、输出和审计这些硬层面，而不是寄希望于模型“自己守规矩”。** <sup>28</sup>

建议的 Security & Governance Plane 组件矩阵如下：

| 组件                | 责任                            | 接口                                | 核心策略                                                      | 存储 / 记录              | 评测                          |
|-------------------|-------------------------------|-----------------------------------|-----------------------------------------------------------|----------------------|-----------------------------|
| Input Sanitizer   | 清洗用户输入、外部文档内容                 | <code>security.input_scan</code>  | injection pattern、URL/domain policy、HTML/script stripping | audit log            | prompt-injection attack set |
| Permission Engine | 基于 workspace/project/user 的授权 | <code>authz.check</code>          | RBAC + scope + tool auth_scope                            | Postgres             | unauthorized access tests   |
| Approval Gate     | 高风险动作审批/编辑/拒绝                 | <code>approval.request</code>     | 按 side_effect_level 决策                                    | audit + decision log | human-in-loop tests         |
| Redaction Engine  | PII/敏感字段脱敏                    | <code>redact(input/output)</code> | regex + classifier + entity tags                          | redaction log        | false pos/neg               |
| MCP Trust Manager | 管理远程 MCP server 信任            | <code>mcp.trust.evaluate</code>   | allowlist、OAuth、server fingerprint、risk tier              | trust registry       | remote MCP pen tests        |
| Credential Broker | 短期凭证下发                        | <code>cred.issue</code>           | least privilege、TTL、audience binding                      | Vault                | token leakage tests         |
| Network Policy    | 工具和沙箱出网控制                     | sandbox egress policy             | deny-by-default、allowlist only                            | firewall/audit       | exfiltration tests          |
| Audit Logger      | 不可抵赖审计                        | append-only events                | every decision/tool/approval                              | object + SQL         | forensic replay             |

沙箱设计建议分三层：**策略沙箱、工作区沙箱、执行沙箱**。策略沙箱先决定“能不能做”；工作区沙箱决定“在哪个目录和哪份资料里做”；执行沙箱再决定“用什么隔离强度去做”。实际执行器建议支持三档：同进程/子进程（仅限只读或纯计算）、rootless Docker/gVisor（中风险命令与文件改写）、microVM/



Firecracker/Kata（高风险代码执行、外部依赖安装、未知脚本）。Docker rootless 官方说明它通过非 root 用户运行 daemon 与容器来降低风险；gVisor 通过 userspace kernel 增强隔离；Firecracker 与 Kata 则通过轻量虚拟机提供更强硬件虚拟化边界。<sup>29</sup>

| 沙箱类型                         | 隔离强度 | 性能 | 适用任务                 | 推荐政策                              |
|------------------------------|------|----|----------------------|-----------------------------------|
| Process sandbox              | 低    | 最高 | 纯本地只读函数、小脚本、格式转换     | 默认禁网、限制 cwd、超时 5-10s              |
| Rootless Docker / gVisor     | 中    | 高  | 受控 CLI、依赖安装、文件编辑、批处理 | rootless、只挂载任务目录、allowlist egress |
| MicroVM / Kata / Firecracker | 高    | 中  | 不可信代码、外部仓库执行、复杂命令    | 强制 approval、短期凭证、只读基础镜像           |

评测与可观测性方面，LangSmith 应当是“统一 trace 母线”，不是“可有可无的外部看板”。官方 run/span 数据格式已经足以支持 Zuno 所需的大多数字段：`trace_id`，`session_id/thread_id`，`run_type`，`inputs/outputs`，`events`，`tags`，`total_tokens`，`total_cost` 等；offline/online evaluation 工作流也支持从人工样本、历史 traces 和 synthetic data 建 dataset，再定义 evaluators、跑 experiments、比较回归。对 Zuno 而言，最好的做法是：业务 trace 先按 LangSmith 兼容 schema 记录，能连 LangSmith 时就同步，不能连时也能先落 SQL/Parquet，再做二次导出。<sup>30</sup>

建议的 Eval & Observability Plane 如下：

| 维度        | 指标                                                                | 工具                               | 说明          |
|-----------|-------------------------------------------------------------------|----------------------------------|-------------|
| Retrieval | recall@k、context recall、BM25 hit、metadata filter hit              | 内建统计 + RAGAS                     | 评估是否漏召回     |
| Ranking   | rerank precision@k、context precision                              | DeepEval / RAGAS                 | 评估 top 排序质量 |
| Answer    | faithfulness、answer relevancy、citation coverage                   | RAGAS / DeepEval                 | 评估是否忠于证据    |
| Tooling   | tool success rate、arg correctness、approval bypass rate            | 内建规则 + DeepEval JSON correctness | 评估工具编排可靠性   |
| Runtime   | step count、replan count、timeout、token/cost                        | LangSmith + metrics store        | 评估运行效率      |
| Security  | prompt injection block rate、redaction miss rate、sandbox violation | 自建 attack suite                  | 评估防护能力      |

其中可以设置一组**可配置目标阈值**，供简历和 CI gate 使用，但这些阈值应当被写成“目标值”，而不是对外吹成“已达成事实”。推荐起步目标如下：

| 指标                  | 建议目标   |
|---------------------|--------|
| retrieval recall@10 | ≥ 0.85 |
| rerank precision@5  | ≥ 0.80 |

| 指标                              | 建议目标   |
|---------------------------------|--------|
| citation coverage               | ≥ 0.95 |
| answer faithfulness             | ≥ 0.90 |
| tool deterministic success rate | ≥ 0.98 |
| prompt injection block rate     | ≥ 0.95 |
| redaction miss rate             | ≤ 0.01 |
| sandbox escape incidents        | 0      |

数据集设计建议同时包含离线与在线两类。LangSmith 官方推荐用人工 curated cases、历史生产 traces、synthetic generation 三种方式建 dataset；RAGAS 与 DeepEval 都支持 reference-based 或 reference-free judge 路线。因此 Zuno 的评测集应至少有四层：**检索集**、**回答集**、**工具集**、**安全集**。检索集覆盖 recall/precision；回答集覆盖 faithfulness/answer relevancy/citation；工具集覆盖参数正确性、side-effect gate 与 JSON schema；安全集覆盖 prompt injection、越权、数据泄露与输出违规。<sup>31</sup>

一个兼容 LangSmith/自建评测的样例 schema 如下：

```
{
  "example_id": "qa_0001",
  "split": "regression",
  "input": {
    "question": "员工手册中试用期请假规则是什么？"
  },
  "expected": {
    "answer_keypoints": [
      "试用期请假需主管审批",
      "病假需提供证明"
    ],
    "citations": ["handbook_v3#p12", "handbook_v3#p13"]
  },
  "metadata": {
    "workspace_id": "ws_hr",
    "query_type": "basic",
    "risk_tier": "low"
  }
}
```

## 部署形态与实施路线

后端部署建议采用“FastAPI API 层 + 长任务 worker 层 + 沙箱池 + 存储平面”的形态。FastAPI 官方非常适合做 typed REST 与 WebSocket；BackgroundTasks 适合轻量级 response-after side work，但长时间、可重试、可横向扩展的任务更适合 Celery 这类 task queue；Deep Agents / LangGraph 负责长任务行为编排；deepsearch-agents 则给出了 FastAPI + WebSocket + DeepAgents + React 的非常贴近你目标的参考实现。<sup>32</sup>

因此，推荐的服务拓扑是：

| 层                  | 推荐实现                                         | 说明                                           |
|--------------------|----------------------------------------------|----------------------------------------------|
| API Gateway        | FastAPI                                      | REST + WebSocket + auth + upload/download    |
| Session/Connection | ConnectionManager                            | thread/session/ws room 管理，前端实时事件推送           |
| Agent Workers      | LangGraph / Deep Agents                      | 执行 Single Controller runtime、subagents、trace |
| Ingest Workers     | Celery/RQ workers                            | 文档解析、embedding、graph extraction、重试任务         |
| Sandbox Cluster    | rootless docker / gVisor / microVM pool      | 代码执行、CLI、受控 SSH                              |
| SQL Store          | PostgreSQL                                   | metadata、audit、memory、approvals、provenance   |
| Vector Store       | pgvector 起步，Milvus/Weaviate 可升级              | retrieval                                    |
| Object Store       | MinIO/S3-compatible                          | 原始文件、产物、导出 trace                             |
| Secrets            | Vault                                        | scoped credentials、TTL token、audit           |
| Observability      | LangSmith-compatible + Prometheus/Grafana 可选 | traces、evals、cost/latency                    |

关于前端，按你的要求，本报告把 **React + Vite** 写为 Target baseline；但必须明确：这不是当前 Zuno 的 Current，因为仓库当前是 Vue Web + Electron Desktop。若你的目标是尽快把架构落到产品，React + Vite 的好处是与 `deepsearch-agents` 的参考形态更接近；若你的目标是最小迁移成本，则应保留当前 Vue/Electron，只把后端与 runtime 升级。这个取舍应在架构文档中明确写成 Target decision，而不能和 Current 混写。 <sup>33</sup>

实施路线建议分七个 Phase，每一阶段都带交付物、测试、verifier 和 stacked PR 边界。

| Phase   | 目标                                                   | 主要交付物                                                            | 核心测试                                  |
|---------|------------------------------------------------------|------------------------------------------------------------------|---------------------------------------|
| Phase 1 | 目标架构骨架与契约落地                                          | docs/architecture、runtime contracts、ToolCard schema、trace schema | schema tests、docs sync verifier       |
| Phase 2 | Document Ingestion + Basic RAG                       | parser router、OCR fallback、chunk/embed/index、basic retrieval API | parser golden tests、recall baseline   |
| Phase 3 | Memory & Context Builder + Single Controller runtime | context packer、memory stores、plan/react/reflect loop、WS events   | loop determinism、context budget tests |

| Phase   | 目标                             | 主要交付物                                                                          | 核心测试                                        |
|---------|--------------------------------|--------------------------------------------------------------------------------|---------------------------------------------|
| Phase 4 | Tool Plane + Policy + Approval | ToolCard registry、executors、policy engine、approval flow、credential broker stub | tool contract tests、approval path tests     |
| Phase 5 | GraphRAG + Evidence/ Citation  | graph extraction、community reports、local/global/DRIFT router、citation builder  | graph query tests、citation coverage         |
| Phase 6 | Eval & Observability           | LangSmith spans、dataset pipeline、RAGAS/DeepEval、CI regression gate             | offline/online eval jobs、trace completeness |
| Phase 7 | Deployment hardening           | task queue、sandbox cluster、secrets、rate limits、frontend workspace、ops docs     | e2e load tests、security drills              |

这一分期方式也与 Zuno 当前仓库的“program/archive/verifier”治理方式相容：当前仓库已经把 architecture render check、Agent/doc/repo hygiene、workflow verifier 这些治理机制跑起来，并把大程序归档到 docs/history/programs。因此后续最好的做法不是重新发明流程，而是让新的 Target Architecture program 继续沿用 stacked PR。<sup>2</sup>

建议的 stacked PR 方式如下：

| 分支                       | 基底     | PR 内容                            |
|--------------------------|--------|----------------------------------|
| codex/zuno-target-phase1 | main   | docs/contracts/schemas/verifiers |
| codex/zuno-target-phase2 | phase1 | ingestion/basic-rag              |
| codex/zuno-target-phase3 | phase2 | runtime/memory/context/ws        |
| codex/zuno-target-phase4 | phase3 | tools/policy/approval            |
| codex/zuno-target-phase5 | phase4 | graphrag/evidence/citation       |
| codex/zuno-target-phase6 | phase5 | eval/langsmith/ci                |
| codex/zuno-target-phase7 | phase6 | deployment/frontend/hardening    |

验收条件建议统一成七条：

| 验收项                       | 判定标准                                                |
|---------------------------|-----------------------------------------------------|
| Agent self-start          | 新 Agent 仅通过 AGENTS.md + .agent/reference 可找到入口与同步规则 |
| Workflow self-maintenance | 长期规则可沉淀为 docs/templates/change-log                  |
| Docs / HTML sync          | docs/architecture 与 architecture.html 自动校验一致        |
| Runtime traceability      | session/thread/trace/tool/evidence 全链路可追踪           |
| Retrieval quality         | 达到配置阈值，且回归集不退化                                      |
| Safety controls           | approval/redaction/network/sandbox/MCP trust 生效     |

| 验收项      | 判定标准                  |
|----------|-----------------------|
| CI green | 单元、契约、集成、eval gate 全绿 |

对于长耗时任务，多线程/多 worker 的建议是一致的：用队列和子任务，不要把一切塞进单个 HTTP 请求；用 **bounded concurrency**，不要无限并发；用每任务独立 **workspace**，不要共享脏目录；用 **idempotency key** 与 **cancel token**，不要让临时失败导致重跑污染。Deep Agents 官方已经把 subagents 区分为同步与异步，并明确支持 long-running、parallel workstreams、mid-flight steering 和 cancellation；Celery 也支持多 worker、命名 worker 与横向扩展。 <sup>34</sup>

## Codex 与自动化执行提示词

下面给出的提示词是“把本报告变成程序/PR”的最短路径模板。它们都应该以“**不得把 Target/Future 写成 Current；修改 docs 时必须同步 architecture.html；所有新增 contract 必须带测试**”为前置规则，这与当前 Zuno 仓库的治理方式保持一致。 <sup>7</sup>

### Phase 1 提示词：架构骨架、契约与文档同步

你现在的在 Zuno 仓库内工作。目标是建立 enterprise-private-knowledge-agent-workspace-v1 的 Phase 1。

要求：

1. 不改变 Current 事实描述；所有新增内容写入 Target Architecture 文档。
2. 新增 docs/architecture/target-enterprise-private-knowledge-workspace.md。
3. 新增 docs/architecture/diagrams/\*.md，分别放置 10 张 Mermaid 图源码。
4. 新增 backend/contracts/ 下列 schema：
  - context\_pack.py
  - task\_plan.py
  - trace\_event.py
  - tool\_card.py
  - evidence\_record.py
5. 新增 tests/contracts/\*，覆盖上述 schema 的 roundtrip 与 required fields。
6. 更新 architecture.html 的生成源，并确保 render 脚本通过。
7. 更新 .agent/reference/architecture-docs-map.md 和 diagram-inventory.md。

输出：

- 变更文件清单
- 关键 schema 摘要
- 运行过的测试与结果

### Phase 2 提示词：Document Ingestion 与 Basic RAG

在上一阶段基础上实现 Phase 2: Document Ingestion Plane + Basic RAG。

要求：

1. 新增 backend/ingestion/：
  - parser\_router.py
  - parsers/docling\_parser.py
  - parsers/unstructured\_parser.py
  - parsers/paddleocr\_parser.py
  - chunking/layout\_semantic.py
  - provenance\_store.py

2. 新增 backend/retrieval/basic\_rag/:
  - ingest\_service.py
  - hybrid\_retriever.py
  - rerank\_pipeline.py
3. 定义 ParseRequest / DocumentAST / ChunkRecord / IngestJob schema。
4. 支持 pdf/docx/pptx/md/txt/html/code/images 的路由与 fallback。
5. 新增测试:
  - parser golden tests
  - chunk coverage tests
  - retrieval smoke tests
6. 新增 verifiers:
  - supported-format verifier
  - provenance-required verifier
7. 更新文档与 Mermaid 图 5、6、7。

### Phase 3 提示词: Single Controller Runtime、Memory、Context Builder

在上一阶段基础上实现 Phase 3: Single Controller Agent Runtime。

要求:

1. 新增 backend/runtime/:
  - state.py
  - context\_builder.py
  - planner.py
  - react\_loop.py
  - reflection.py
  - replan.py
  - reflexion.py
  - session\_manager.py
2. 用 LangGraph 风格状态机表达 ReceiveGoal -> SafetyCheck -> BuildContext -> Plan -> ExecuteStep -> Reflect -> Replan -> AnswerDraft -> FinalReflect -> MemoryWrite -> TraceCommit。
3. 新增 backend/memory/:
  - stores/conversation\_store.py
  - stores/project\_store.py
  - stores/user\_store.py
  - stores/episodic\_store.py
  - stores/procedural\_store.py
  - compression/summary.py
  - compression/importance\_filter.py
  - compression/structured\_extract.py
4. 暴露 API:
  - POST /v1/sessions
  - POST /v1/agent/turns
  - WS /v1/sessions/{session\_id}/events
5. 新增测试:
  - runtime state transition tests
  - context budget tests
  - memory compression tests

- ws event sequence tests
6. 更新文档与 Mermaid 图 3、7、9。

#### Phase 4 提示词: Tool Plane、Policy、Sandbox、Approval

在上一阶段基础上实现 Phase 4: Tool / Capability Plane + Security Controls。

要求:

1. 新增 backend/tools/:
  - registry/tool\_registry.py
  - router/tool\_router.py
  - policies/policy\_engine.py
  - executors/local\_function\_executor.py
  - executors/sdk\_executor.py
  - executors/api\_executor.py
  - executors/cli\_executor.py
  - executors/ssh\_executor.py
  - executors/mcp\_stdio\_executor.py
  - executors/mcp\_http\_executor.py
2. 新增 backend/security/:
  - approval\_service.py
  - credential\_broker.py
  - redaction.py
  - prompt\_injection\_guard.py
  - network\_policy.py
3. 新增 backend/sandbox/:
  - base.py
  - process\_executor.py
  - rootless\_docker\_executor.py
  - microvm\_executor\_stub.py
4. 提供至少 6 张 ToolCard 样例:
  - knowledge.search
  - file.read
  - file.search\_workspace
  - email.send\_draft
  - db.query\_readonly
  - ssh.exec\_safe
5. 新增测试:
  - policy path tests
  - approval interrupt tests
  - tool schema tests
  - sandbox deny/allow tests
6. 更新文档与 Mermaid 图 4、8、10。

为了让 Codex/automation 持续稳定地产出 stacked PR, 建议再补两类模板: 一类是“文件生成模板”, 一类是“多 worker 脚本模板”。

#### Mermaid 文件模板

请为 docs/architecture/diagrams/<name>.md 生成内容：

- 标题
- 图的用途
- Current / Target / Future 边界说明
- Mermaid 源码
- 与代码目录的映射
- 与测试/trace/eval 的对应关系

禁止：

- 把未实现能力写成 Current
- 使用与仓库目录不一致的模块名

## ToolCard 模板

请为 backend/tools/cards/<tool\_id>.json 生成 ToolCard。

必填字段：

tool\_id, name, display\_name, domain, description, connector\_type, executor\_type, input\_schema, output\_schema, read\_only, side\_effect\_level, approval\_mode, trust\_tier, auth\_scope, budget\_policy, observability

同时生成 tests/tools/test\_<tool\_id>\_card.py 用于 schema 校验。

## Sandbox 执行器模板

请为 backend/sandbox/<executor>.py 生成一个可测试的执行器 stub：

- 统一接口 execute(command, cwd, env, timeout, network\_profile, mount\_profile)
- 返回 exit\_code/stdout/stderr/artifacts
- 内置审计事件 emit\_audit\_event(...)
- 任何危险命令先走 policy hook
- 所有实现必须可被 mock 测试

## 多线程 worker / 长任务脚本模板

请新增 scripts/run\_parallel\_ingest.py 与 backend/workers/supervisor.py

要求：

- 使用任务队列或任务抽象，不在 HTTP handler 内做重任务
- 支持 bounded concurrency
- 任务有 idempotency\_key
- 任务支持 cancel\_token
- 每个任务创建独立 workspace
- 把 thread\_id / trace\_id 透传到子任务
- 失败重试不重复写入最终索引

最后，Git 策略建议保持非常机械化：**一阶段一分支、一主题一 PR、每个 PR 都可单独过 verifiers、严禁把 architecture/docs/test/implementation 混成无法 review 的巨型提交**。如果一个阶段内部还包含三个明显主题，就再拆成 phaseX-a / phaseX-b / phaseX-c 的 stacked 分支。这样做的目的不是“流程好看”，而是让 Agent、自查脚本、评测与人工 code review 都能在一致边界内工作。这个建议也与当前 Zuno 已形成的 phased programs 与归档习惯高度一致。 <sup>2</sup>



- 
- 1 2 7 33 <https://github.com/ProfessorZhi/Zuno/blob/main/README.md>  
<https://github.com/ProfessorZhi/Zuno/blob/main/README.md>
- 3 6 13 <https://microsoft.github.io/graphrag/>  
<https://microsoft.github.io/graphrag/>
- 4 <https://github.com/didilili/deepsearch-agents>  
<https://github.com/didilili/deepsearch-agents>
- 5 9 <https://docs.langchain.com/oss/python/langgraph/overview>  
<https://docs.langchain.com/oss/python/langgraph/overview>
- 8 <https://github.com/ProfessorZhi/Zuno/blob/main/docs/architecture.html>  
<https://github.com/ProfessorZhi/Zuno/blob/main/docs/architecture.html>
- 10 21 <https://arxiv.org/abs/2210.03629>  
<https://arxiv.org/abs/2210.03629>
- 11 22 **Tools**  
[https://modelcontextprotocol.io/specification/2025-03-26/server/tools?utm\\_source=chatgpt.com](https://modelcontextprotocol.io/specification/2025-03-26/server/tools?utm_source=chatgpt.com)
- 12 26 **Docling integration - Docs by LangChain**  
[https://docs.langchain.com/oss/python/integrations/document\\_loaders/docling?utm\\_source=chatgpt.com](https://docs.langchain.com/oss/python/integrations/document_loaders/docling?utm_source=chatgpt.com)
- 14 19 <https://arxiv.org/html/2309.02427v3>  
<https://arxiv.org/html/2309.02427v3>
- 15 <https://docs.langchain.com/oss/python/deepagents/customization>  
<https://docs.langchain.com/oss/python/deepagents/customization>
- 16 30 <https://docs.langchain.com/langsmith/run-data-format>  
<https://docs.langchain.com/langsmith/run-data-format>
- 17 32 <https://fastapi.tiangolo.com/>  
<https://fastapi.tiangolo.com/>
- 18 **Models - Docs by LangChain**  
[https://docs.langchain.com/oss/python/deepagents/models?utm\\_source=chatgpt.com](https://docs.langchain.com/oss/python/deepagents/models?utm_source=chatgpt.com)
- 20 <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>  
<https://platform.claude.com/docs/en/build-with-claude/prompt-caching>
- 23 **langchain-ai/deepagents: The batteries-included agent ...**  
[https://github.com/langchain-ai/deepagents?utm\\_source=chatgpt.com](https://github.com/langchain-ai/deepagents?utm_source=chatgpt.com)
- 24 <https://arxiv.org/abs/2005.11401>  
<https://arxiv.org/abs/2005.11401>
- 25 <https://docs.weaviate.io/weaviate/concepts/search/hybrid-search>  
<https://docs.weaviate.io/weaviate/concepts/search/hybrid-search>
- 27 **Welcome to Faiss Documentation — Faiss documentation**  
[https://faiss.ai/index.html?utm\\_source=chatgpt.com](https://faiss.ai/index.html?utm_source=chatgpt.com)
- 28 <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>  
<https://genai.owasp.org/llmrisk/llm01-prompt-injection/>
- 29 <https://docs.docker.com/engine/security/rootless/>  
<https://docs.docker.com/engine/security/rootless/>

<sup>31</sup> <https://docs.langchain.com/langsmith/evaluation>

<https://docs.langchain.com/langsmith/evaluation>

<sup>34</sup> <https://docs.langchain.com/oss/python/deepagents/subagents>

<https://docs.langchain.com/oss/python/deepagents/subagents>